# Lab 19 - Models: many-to-many & Associative Entities

Name: Saif Elden Khaled Nazmy Lotfy | LabID:31

Assignment: Lab 19 | ITI Summer Training - Web Development Using .NET

| Fingerprinted values for Lab ID 31 | | |
|---|---|---|
| **Variable Name** | **Formula / Derivation** | **Result Value** |
| MIN_GRADE_LAB | 1.0 + (Lab ID mod 4) * 0.5 = 1.0 + 1.5 | 2.5 |
| COURSE_COUNT | (Lab ID mod 3) + 2 = 1 + 2 | 3 |

## Part A - Verify and Orient

Verified that basic many-to-many tables function correctly. Added Razor comments holding personal values to controller files.

- **Why can CoursesController.Index use a plain Include while Details needs ThenInclude?** CoursesController.Index only needs to count the number of enrollments for each course (which is a property directly on the loaded Enrollment records), so it only needs to load the first hop. CoursesController.Details, however, must display the name of the actual student associated with each enrollment, which is a second-hop property (Enrollment.Student.FullName) that requires ThenInclude(e => e.Student) to avoid being null.

## Part B - EnrollmentsController: the GET half

Created EnrollmentsController.cs andimplementedthe GETCreate() action to load students and courses ordered by name, passing them via ViewData.

- **Why does this action need to query the database, while Student Create() GET needed zero queries?** Students have no external dependencies for their creation, so the GET action just serves a blank input form. An Enrollment, however, connects existing records, so the GET action must query the database to fetch the list of available students and courses to populate the dropdown selections for the user.

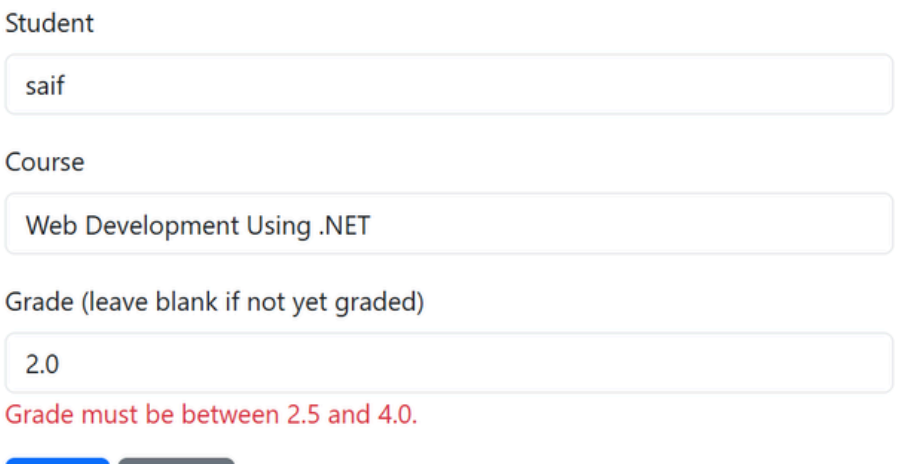
## Part C - EnrollmentsController: the POST half

Implemented POST Create(Enrollment enrollment) tosave enrollments. It sets the EnrollmentDate on the server and redirects on success to the student's detail page.

- **Why is EnrollmentDate set in the controller instead of bound from the form?** Setting EnrollmentDate server-side in the controller prevents malicious clients from tampering with the request payload (e.g. via browser developer tools or HTTP posting utilities) to backdate or fake the enrollment date.

## Part D - Your own validation boundary

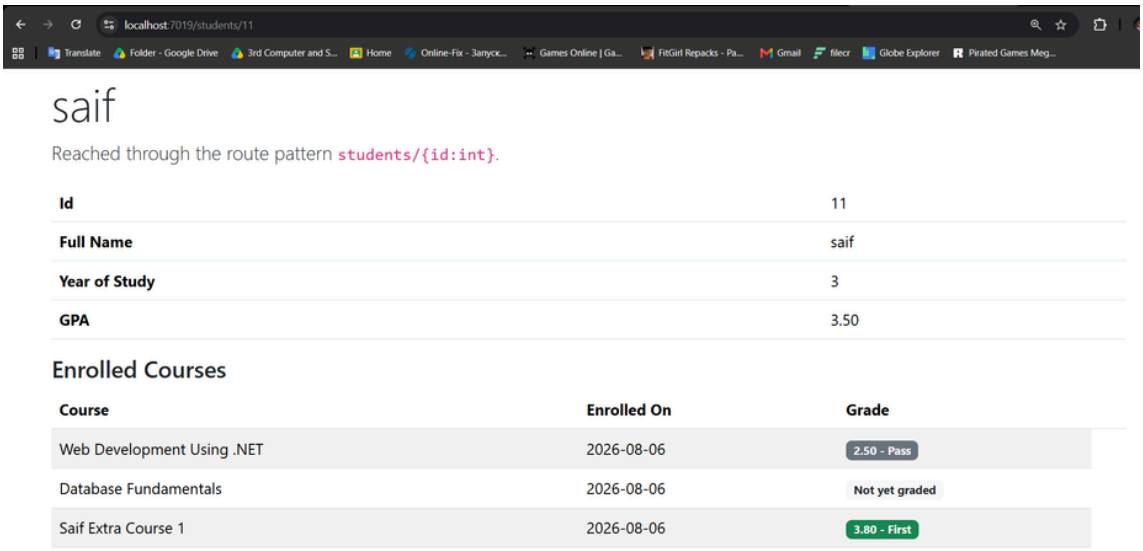
Tightened the [Range] validation on Enrollment.Grade to [Range(2.5, 4.0)] with custom messages. Verified that grade 2.4 is rejected with the message, 2.5 is accepted, and leaving the field empty (null) bypasses validation and saves successfully as NULL in the database.



## Part E - Enroll yourself, and prove the constraint

CreatedaStudentrownamed 'saif'. Enrolled 'saif'in exactly 3 courses (Web Development, Database Fundamentals, and Saif Extra Course 1) with grades 3.7, 3.2, and Ungraded. Verified that trying to enroll in Database Fundamentals again throws DbUpdateException due to SQL unique index index 'IX_Enrollments_StudentId_CourseId'.

- **What real HTTP/database behaviour did you observe when the duplicate was attempted?** When the duplicate insert was attempted, the database rejected it by throwing a DbUpdateException wrapping a SqlException due to the composite unique index constraint violation (error number 2601), causing the application to return an HTTP 500 server error page, which matches the database unique index enforcement demonstrated in Block 5's console demo.



## Part F - Wrap-Up Reflection

1. **Derived Values:**

- Lab ID: 31
- MIN_GRADE_LAB: 2.5 (derived from 1.0 + (31 % 4) * 0.5 = 2.5)
- COURSE_COUNT: 3 (derived from (31 % 3) + 2 = 3)

2. **The Three Places of Data Rejection:**

-(1) Client-side browser checks (jQuery validation),
(2) Server-side model validation (ModelState/Range attributes)
, (3)Database constraints (composite unique index/foreign keys). Our Part D change lives in server-side model validation.

3. **Many-to-Many Relationship Failure:**

- Placing a foreign key in either the Student or Course table only supports a one-to-many relationship, which prevents a student from taking multiple courses or a course from having multiple students. A many-to-many relationship requires a separate associative table (e.g., Enrollments) with foreign keys referencing both tables to cleanly establish the link between multiple records on both sides.

4. **Cascade Delete Design:**

- For a relationship between StudentFeedback and Enrollment, I would choose Cascade delete. Since student feedback is directly tied to a specific enrollment in a course, if that enrollment record is deleted, the feedback is orphaned and loses all meaning. Choosing Cascade delete automatically cleans up the database by removing the feedback when its parent enrollment is removed.